



The Speech Coaching Platform

How the whole system works, why you chose it, and how to explain it without sounding like a computer manual.

Ask → Check → Store → Queue → Work → Show

The one-minute version

A speaker uploads a private video, invites specific people, and receives feedback tied to exact moments. The system keeps the video safe, prepares it in the background, and shows the right note at the right second.

EXPLAIN IT TO A 10-YEAR-OLD

Imagine a private school theater. The **speaker owns the stage**. Invitations are tickets. A reviewer gets one folder for that performance and puts sticky notes at exact times: "Great pause at 1:23."

TECHNICAL TRANSLATION

A React single-page app calls a Laravel JSON API. PostgreSQL stores relationships and permissions; SeaweedFS stores media; Valkey queues work; FFmpeg and Whisper workers process video and captions.

The six things to remember

FRONT DESK

React

Draws the screens and player in the browser.

PRINCIPAL

Laravel

Checks identity, rules, and what happens next.

LEDGER

PostgreSQL

Stores facts, links, states, and hard rules.

WAREHOUSE

SeaweedFS

Stores the big video, picture, audio, and caption files.

TASK BOARD

Valkey

Holds jobs that should happen in the background.

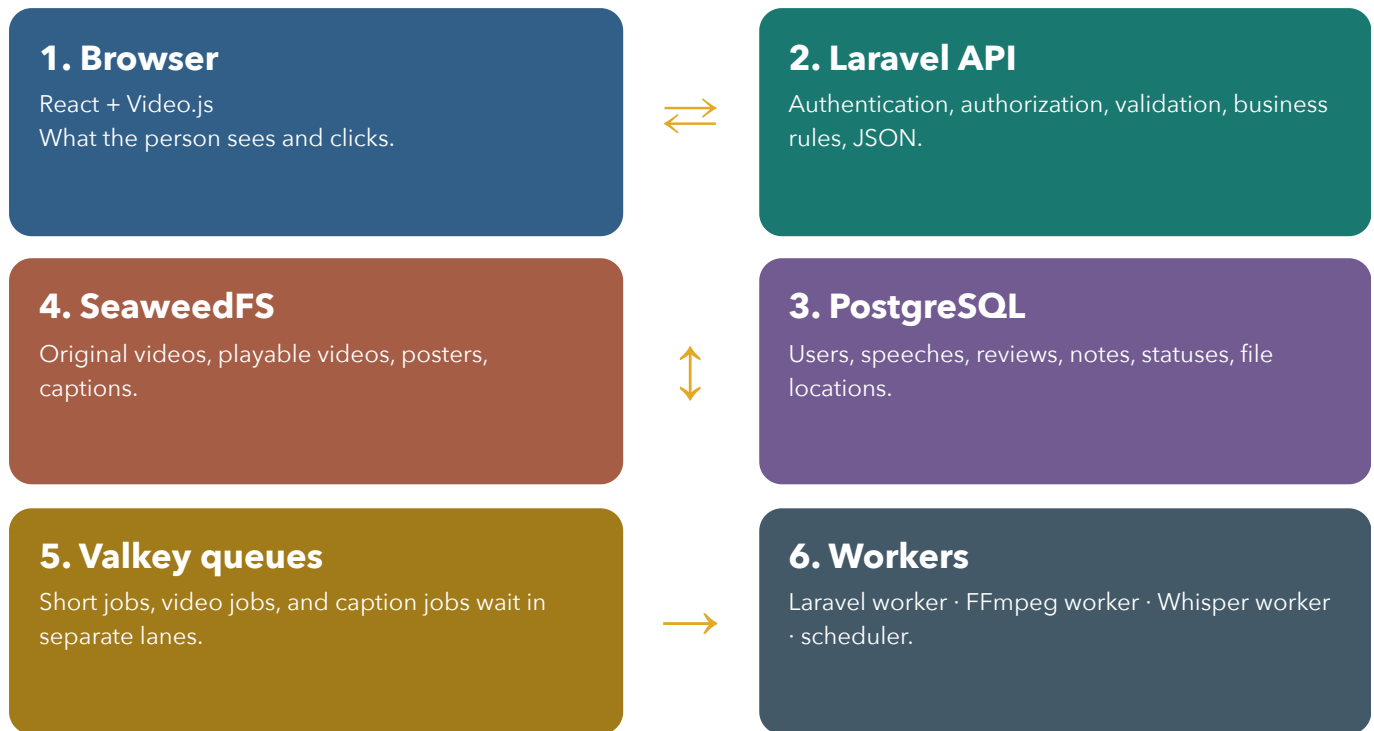
ROBOTS

Workers

Convert video, make captions, send mail, and repair stale work.

The database describes the speech. Object storage holds the speech. Laravel decides who may touch either.

The architecture in one picture



The surprising shortcut: large video bytes do not travel through Laravel. Laravel gives the browser a temporary, narrowly scoped permission slip. The browser then uploads to or watches from SeaweedFS directly.

This is a modular monolith—not microservices

There is **one Laravel application and one business model**. The separate containers are different jobs or infrastructure, not independent mini-products with separate databases and APIs. That keeps reasoning simple while isolating heavy work.

EXPLAIN IT TO A 10-YEAR-OLD

One school has many rooms: office, library, warehouse, workshop. It is still one school.

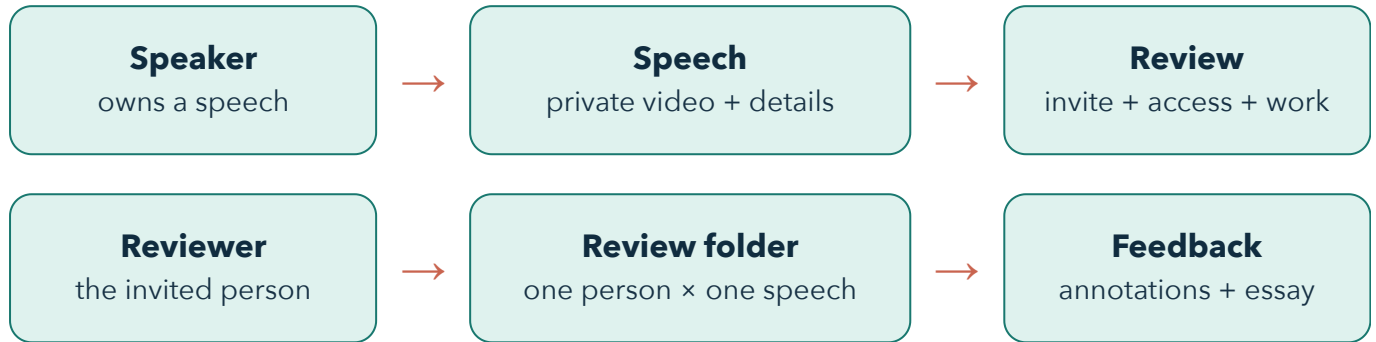
TECHNICAL TRANSLATION

One deployable domain codebase serves HTTP and queue commands. Process boundaries isolate workload and dependencies.

The production web container serves the built React files with Nginx and forwards API requests to PHP-FPM. During development, Vite runs on the host for fast refresh.

The most important design decision

The **Review** row is both the reviewer's permission to enter and the folder containing that reviewer's work.



EXPLAIN IT TO A 10-YEAR-OLD

A theater ticket has a person's name and one show on it. The ticket opens the door. The same ticket number is printed on that person's feedback folder. No ticket means no folder and no notes.

Why this shape is powerful

- ✓ An annotation must belong to a review, so an unauthorised "orphan note" cannot exist.
- ✓ One database rule allows only one review per person per speech.
- ✓ Revoking the review removes the access grant at the same boundary that owns the feedback.
- ✓ Reviewers cannot see one another's folders or even use the reviewer count as a clue.

Make wrong states impossible to represent, not merely forbidden in a document.

Core cast of data

THING	WHAT IT MEANS	WHAT HANGS FROM IT
User / Profile	An account and public identity.	Speeches, review invitations, notifications.
Speech	One private speaking attempt.	Assets, transcript, reviews, previous-attempt link.
SpeechAsset	A file record and processing status.	Source, playable video, poster, captions.
Review	One reviewer's grant and body of work.	Timed annotations and one essay.

Who can see what?

Every speech is private. There is no “public” switch and no open pool where strangers can claim a speech.

PERSON	MAY WATCH?	MAY READ FEEDBACK?	MAY WRITE?
Speaker	Own speech	Published reviewer work	Edits own captions; invites
Invited reviewer	Only after the grant reaches an access-giving state	Only their own review	Only their own review
Unrelated member	No	No	No
Admin	For moderation	For moderation	No coaching feedback

How the check happens



EXPLAIN IT TO A 10-YEAR-OLD

A session is a coat-check ticket matched against a live guest list. Cross your name off the list and your next request is refused.

TECHNICAL TRANSLATION

Sanctum/Fortify use stateful SPA sessions. Policies enforce record-level access; role checks handle capabilities.

Why sessions instead of JWTs?

This product often needs to **take access away now**: revoke a reviewer, suspend an account, or remove a role. A session is checked against server state on every request. A self-contained JWT remains valid until it expires.

Important nuance: a media link already issued is temporary, not magically recallable. Its short lifetime limits the remaining window, and Laravel refuses to issue a new link after access is revoked.

What happens in the browser

React

Draws pages as components. When state changes, it updates only the necessary screen pieces.

screens

components

React Router

Maps addresses such as `/speeches/:id` to the right page and protected layout.

navigation

Redux Toolkit

Holds shared screen state—the small facts several components need to agree on.

shared UI state

RTK Query

Fetches server data, caches it, reports loading/errors, and refreshes labelled data after changes.

server cache

API calls

Video.js

Wraps the browser video player and gives the app a consistent player adapter.

playback

Tailwind + UI components

Supplies design tokens, responsive styling, controls, and accessible primitives.

presentation

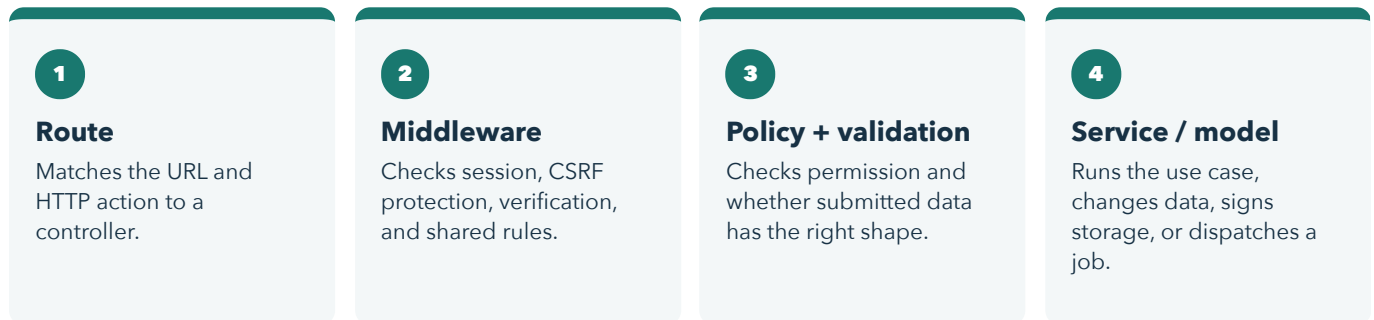
The three-bucket state rule

PUT IT HERE	WHEN	EXAMPLE
RTK Query	The server owns the truth.	Speech, reviews, captions, transcript.
Redux / route state	Several UI pieces need the same choice.	Selected reviewer track.
Local React / player state	It changes very quickly or belongs to one component.	Typing draft, current video time.

Crucial performance choice: the playhead can change dozens of times per second. It stays out of Redux, so the whole page does not redraw on every frame. Think of a shared class whiteboard: a child tapping a pencil 60 times a second does not report every tap to the whiteboard keeper.

What Laravel does

Laravel is the rule-keeper and coordinator. It should decide and record—not carry 200 MB videos through the office.



A concrete example: “Invite Jordan”

1. The browser sends a request to create a review for a speech.
2. Authentication proves who is making the request.
3. The speech policy proves that person owns the speech.
4. Validation proves Jordan is a valid reviewer and not a duplicate.
5. The review service creates the invitation in a transaction.
6. Laravel queues a notification and returns a safe JSON resource.

Seams are intentional

TECHNICAL TRANSLATION

Controllers stay thin: HTTP details in, response out. Reusable business work sits in services.

TECHNICAL TRANSLATION

Policies stay central: a second screen cannot accidentally invent a different access rule.

TECHNICAL TRANSLATION

Storage is abstracted: app code names a disk/path rather than hard-coding SeaweedFS everywhere.

TECHNICAL TRANSLATION

Jobs are explicit: slow operations have statuses, retries, failure codes, and recovery.

Safe responses: API resources decide what leaves the server. Internal paths, command output, and detailed worker failures do not belong in user-facing JSON.

Why PostgreSQL and object storage have different jobs

LEDGER

PostgreSQL

Excellent at relationships and questions:

- Who owns this speech?
- Did this reviewer accept?
- Which notes are published?
- Is this the one primary video?

WAREHOUSE

SeaweedFS

Excellent at large byte objects:

- Original upload
- Playable MP4
- Poster image
- WebVTT caption file

EXPLAIN IT TO A 10-YEAR-OLD

A library catalogue holds the title, owner, and shelf number. The actual giant book stays on the shelf. Stuffing the book into the index-card drawer would ruin the catalogue.

Why video does not live in PostgreSQL

- ✓ Video seeking needs byte-range reads: "give me this small slice now." Object storage speaks that language.
- ✓ A large video read should not throw useful database pages out of memory.
- ✓ Database backups stay focused on valuable relational facts instead of huge replaceable media bytes.

What PostgreSQL protects even if app code slips

Foreign keys

A note cannot point to a review that does not exist.

Unique rules

One reviewer cannot get two folders for one speech.

Checks + partial indexes

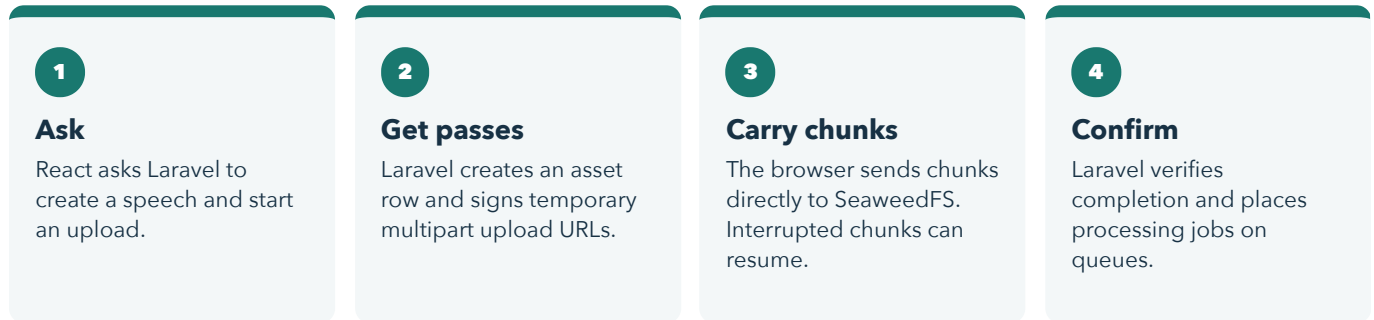
Only valid states and one primary asset of each kind.

The trade-off: the database write and file write cannot be one transaction. Asset statuses such as *uploading*, *processing*, *ready*, and *failed* make that gap visible and repairable.

JOURNEY 1

Uploading a video

Laravel authorizes and coordinates. Uppy and the browser carry chunks directly to SeaweedFS.



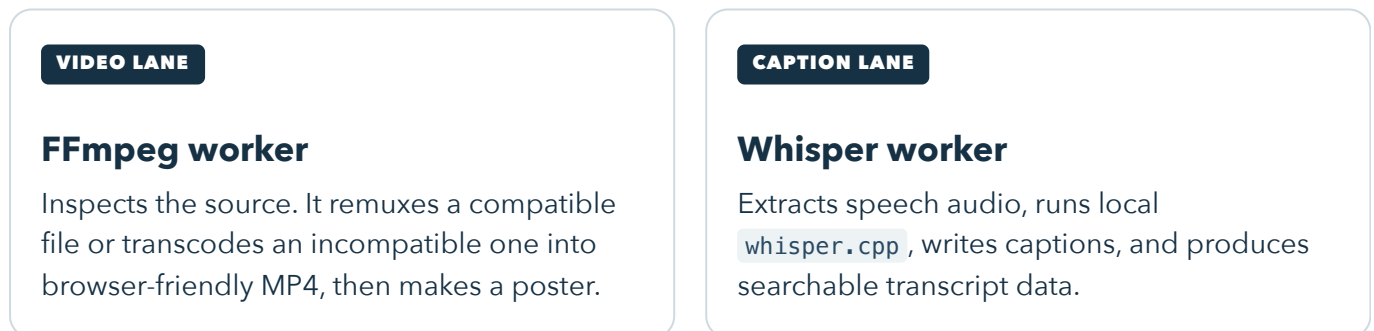
EXPLAIN IT TO A 10-YEAR-OLD

The principal writes a hall pass. The moving company carries boxes straight to the warehouse. The principal records that the delivery arrived but never lifts every box.

Why multipart upload?

A 200 MB video is split into pieces. If hotel Wi-Fi fails after piece 17, only unfinished pieces need to be retried. The server also avoids request-size and memory bottlenecks.

Then two lanes begin



User-experience win: the playable video can become ready while captions are still processing. Slow transcription does not hold the video hostage.

uploading → processing → ready or failed + retry

Watching a private video

1

Open speech

React asks the API for speech information.

2

Check grant

Laravel confirms owner, active review access, or moderation authority.

3

Sign URL

Laravel returns a short-lived storage URL for the ready video.

4

Stream

The browser requests only the byte ranges needed for play and seek.

EXPLAIN IT TO A 10-YEAR-OLD

The principal checks your ticket and gives you a temporary key to one locker. The key cannot open the whole warehouse and soon stops working.

TECHNICAL TRANSLATION

The API returns a presigned GET URL. SeaweedFS serves HTTP Range requests directly to the browser's `<video>` element.

Why Range matters

Dragging from 00:10 to 08:45 should not download the first eight minutes again. The player asks storage for the byte section around 08:45. That makes scrubbing responsive and is a major reason media bytes belong in object storage.

When the temporary URL expires

The player adapter can request a fresh authorized URL and restore playback state. A refresh is another chance for Laravel to deny access if the review was revoked.

Security boundary: a signed URL is a short-lived capability. Treat it like a temporary key: limit its object, method, and lifetime; never log or expose it unnecessarily.

What Laravel does not do

No video proxy

PHP is not continuously relaying the stream.

No public bucket

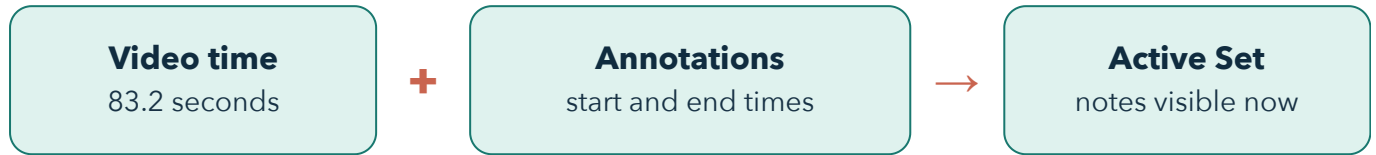
Storage objects are not simply open to everyone.

No permission cache

Revocation checks should use current authority.

Timed feedback: the heart of the product

Each annotation has a start, duration, text, and review. The player shows it only while its time window is active.



EXPLAIN IT TO A 10-YEAR-OLD

A teacher has sticky notes saying “show from 1:20 to 1:26.” A tiny clock helper checks the stopwatch and holds up exactly the notes whose time has arrived.

Two timing signals, one answer

Browser TextTrack

The native timed-text machinery reports active cues accurately when the browser behaves.

Pure reconciler

`computeActive(cues, currentTime)`
independently calculates the active IDs several times per second.

The overlay renders from the reconciled active set. CSS handles the fade, which lets the browser’s compositor do smooth visual work without turning animation into application state.

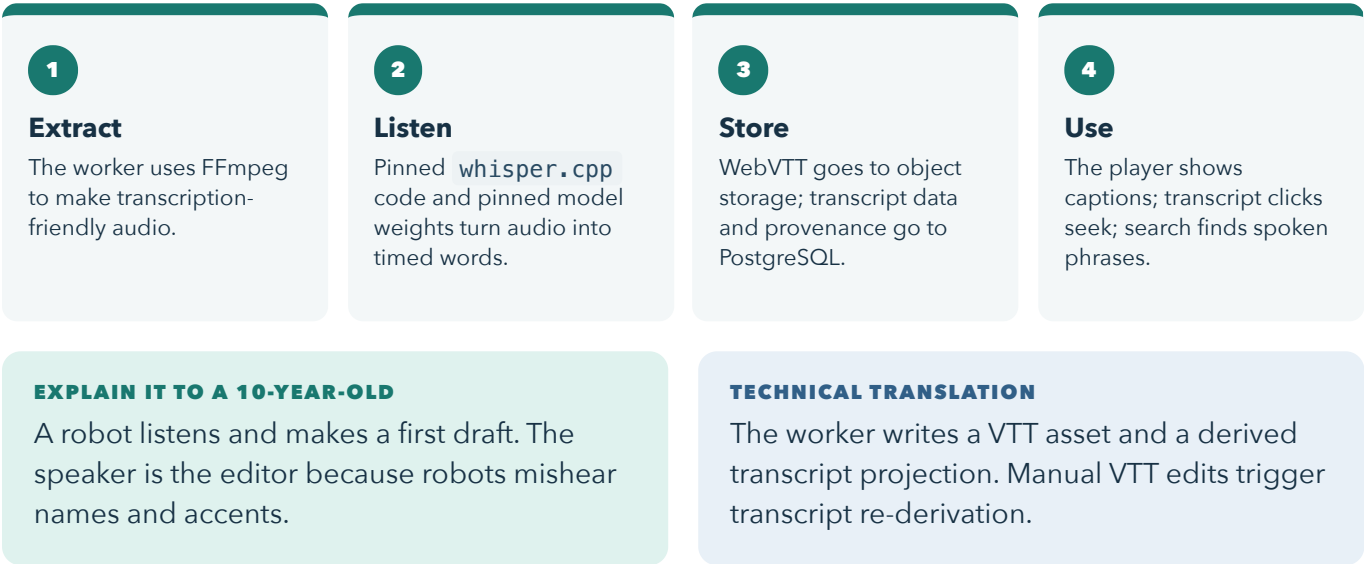
Why the pure function matters

- ✓ It can be tested with numbers—no real video, network, or browser timing required.
- ✓ Seeking backward, overlap, exact boundaries, and bad inputs become cheap test cases.
- ✓ The browser integration becomes a small adapter around proven arithmetic.

Authoring loop: a reviewer pauses at a moment, writes, autosaves a draft, previews the same overlay the speaker will see, then publishes the review as a coherent snapshot.

Captions and transcript

Captions are automatic by default, editable by the speaker, and processed locally so speech recordings do not need to go to a transcription vendor.



Why keep both VTT and transcript data?

REPRESENTATION	BEST AT	LIVES IN
WebVTT file	Native player captions with time cues.	SeaweedFS
Transcript projection	Search, transcript panel, click-to-seek, later analysis.	PostgreSQL

Why pin the engine and model?

The same audio can produce different words after an invisible model upgrade. The system records model/engine provenance so changes are deliberate and results are explainable.

Owner control: the caption setting is an owner-only switch. Disabling automatic generation must not silently delete an already useful, ready caption file.

Why queues and workers exist

A web request should answer in seconds. Video conversion and speech recognition can take minutes, so they become jobs.

EXPLAIN IT TO A 10-YEAR-OLD

A restaurant cashier takes your order quickly and puts a ticket on the rail. The soup cook, pizza cook, and dessert cook each pull from the right lane. You do not stand at the cash register while the pizza bakes.

Three lanes with different clocks

LANE	WORKER	TYPICAL WORK	WHY SEPARATE?
default	queue-worker	Notifications and short projection work.	Fast jobs should stay fast.
transcode	ffmpeg-worker	Probe, remux/transcode, poster.	Long timeout; FFmpeg dependency.
captions	whisper-worker	Audio extraction and transcription.	CPU-heavy; model and CLI dependency.

Reliable-job vocabulary

Idempotent

Running the same job twice should not create two primary videos or corrupt the result.

Attempt identity

An old caption worker must not overwrite a newer retry or manual edit.

Visible status

The UI can show uploading, processing, ready, or a safe failure code.

Reconciliation

The scheduler detects work stranded by a killed worker and repairs or fails it safely.

Key principle: queues do not guarantee “exactly once.” Correctness comes from database constraints, current-attempt checks, transactions, and safe repetition.

Why so many containers?

A container is a repeatable lunchbox: the program plus the tools it needs. Multiple lunchboxes do not automatically mean microservices.

SERVICE	CHILD-FRIENDLY JOB	REAL JOB
web	Front door	Nginx serves the built SPA and forwards API requests.
app	Principal's office	PHP-FPM runs Laravel web requests.
postgres	Ledger	Relational source of truth and constraints.
seaweedfs	Warehouse	S3-compatible object storage.
valkey	Task board	Queue transport and cache infrastructure.
queue-worker	Quick helper	Short Laravel jobs.
ffmpeg-worker	Video robot	Media conversion and poster generation.
whisper-worker	Listening robot	Local transcription and captions.
scheduler	Hall monitor	Runs timed cleanup and recovery commands.
mailpit	Practice mailbox	Catches development email without sending it.

Image vs container

EXPLAIN IT TO A 10-YEAR-OLD

An **image** is a lunch recipe. A **container** is one packed lunch made from it. The app, quick worker, and scheduler can use the same recipe but run different commands.

TECHNICAL TRANSLATION

The multi-stage Dockerfile builds dependency, web, runtime, FFmpeg, and Whisper targets. Compose supplies commands, networks, volumes, dependencies, and health checks.

Persistent volumes: containers are replaceable; database, media, and model files live in named volumes so replacing a process does not erase its durable data.

How the design resists mistakes

Authorization at the server

Hiding a button is friendly UI, not security.
Laravel policies must reject the request itself.

Transactions

Changes that belong together either commit together or roll back together.

Database invariants

Foreign keys, uniqueness, checks, and partial indexes guard facts under concurrency.

Optimistic locking

Multi-writer documents use a version number so one tab cannot silently erase newer work.

Safe failure codes

Users receive stable explanations; private paths and worker stderr remain internal.

Recovery sweeps

The scheduler finds stale processing attempts after hard crashes.

Example: two tabs edit the same essay

1

Both read v5

Tab A and Tab B start from version 5.

2

A saves

The database writes A's text and advances to v6.

3

B tries v5

The server sees B is based on an old version.

4

Conflict, not loss

B reloads or resolves; A's work is not silently overwritten.

EXPLAIN IT TO A 10-YEAR-OLD

Two children copy the same worksheet. One hands in a new answer. The second must not erase it using an older copy without being warned.

The frontend makes the experience pleasant. The backend and database make the rules true.

How the architecture is tested

Use the cheapest test that can honestly prove the behavior. Push logic downward; keep browser tests for real wiring.

MANY + FAST

Unit tests

Pure timing arithmetic, parsing, validation helpers, reducers, and small UI behavior.

Vitest / Pest

RULES + DATA

Feature tests

Policies, API responses, transactions, lifecycle states, storage fakes, queue dispatch.

Laravel + Pest

FEW + REAL

Browser tests

Login, upload wiring, invitation journeys, video/caption tracks, seeking, editor integration.

Playwright

One example across the layers

QUESTION	BEST PROOF
Which note IDs are active at 83.2 seconds?	Unit-test <code>computeActive</code> with numbers.
Can an unrelated user fetch annotations?	Backend policy/feature test expects forbidden.
Does a real video element contain a real captions track and seek from transcript?	Playwright in a real browser.
Can the compiled Whisper adapter invoke the pinned binary and model?	Separate real-stack smoke test.

Avoid false confidence: mocking every boundary can prove that mocks agree with the code while the actual browser, object store, queue, or native binary is broken. Heavy real-smoke tests are separated so normal feedback stays quick.

EXPLAIN IT TO A 10-YEAR-OLD

Test a ruler with measurements, a lock with the wrong keys, and the whole bicycle with a real rider. Do not make the rider test every screw.

What you deliberately did not choose

NOT CHOSEN	REASON	CHOSEN INSTEAD
JWT-first browser auth	Immediate revocation is central to private media.	Stateful Sanctum sessions.
Videos in the database	Poor fit for seeking, memory, backups, and large opaque bytes.	SeaweedFS + metadata rows.
Videos through PHP	Would turn the API into an expensive byte pipe.	Presigned direct upload/download.
One giant worker lane	Five-minute transcription would delay two-second jobs.	Short, transcode, and caption lanes.
Microservices / Kubernetes	Operational cost without a current scaling need.	Modular monolith + Compose.
Public speech pool	Would let strangers grant themselves access.	Owner-created invitations only.
Redux-Saga	Extra programming model without needed capability.	RTK Query + listener middleware.
HLS in v1	More pipeline/storage complexity than needed for current scale.	Browser-friendly MP4 + Range.

The honest weak point

Self-hosted FFmpeg and Whisper are limited by one machine's CPU. That is acceptable now. They are already isolated behind queues, so a future managed transcode/transcription service can replace a worker without rewriting the product model.

Scale readiness means knowing which box to replace later—not pretending every box is infinite today.

MEMORY TRICK

Remember A-C-S-Q-W-S

When explaining any feature, walk through the same six verbs.



Ask

The React browser asks Laravel for a permitted action.



Check

Session, validation, and policy checks decide whether it is allowed.



Store

PostgreSQL stores relational facts; SeaweedFS stores large bytes.



Queue

Anything slow becomes a job on the correct Valkey lane.



Work

A specialized worker performs the expensive task and records status.



Show

RTK Query refreshes server truth; React and Video.js present it.

Try it with captions: Ask to upload → Check owner → Store source → Queue captions → Whisper works → Show VTT and transcript.

The second memory trick

Relationship? Ledger. Large bytes? Warehouse. Slow task? Task board. Permission? Principal.

Ready-made explanations

Thirty seconds

"It is a private speech-coaching app. **React** runs the interface, while **Laravel** owns permissions and business rules. **PostgreSQL** stores relationships and statuses, and **SeaweedFS** stores the large media. The browser uploads and streams media directly using short-lived URLs. Slow jobs go through **Valkey** to separate FFmpeg and Whisper workers, so video can be ready before captions. The key data decision is that a Review is both an invitation/access grant and the folder for one reviewer's notes."

For a 10-year-old

"It is a private theater. The speaker owns the show and hands tickets to chosen reviewers. The principal checks every ticket. A ledger remembers who owns what, a warehouse holds the giant videos, and robots prepare the video and type captions. Each reviewer has one folder of sticky notes that pop up at the right time."

If someone asks, "Why this architecture?"

"Each tool has one job it is good at. The database handles rules and relationships; object storage handles huge seekable files; Laravel handles trust; React handles interaction; queues protect web requests from slow work. The boundaries also give us clean places to test and replace heavy infrastructure later."

If someone asks, "Is it microservices?"

"No. It is a **modular monolith**: one Laravel domain and database, with multiple processes and containers. The workers are separated for timeouts, CPU, and dependencies—not because each is its own product."

One-page cheat sheet

QUESTION	ANSWER
What is the product?	Private, invitation-only video feedback tied to exact moments.
What is the frontend?	React SPA, React Router, Redux Toolkit/RTK Query, Video.js, Tailwind.
What is the backend?	Laravel JSON API with session auth, policies, services, jobs, and resources.
Where are facts?	PostgreSQL.
Where are big files?	SeaweedFS object storage.
How do uploads/playback work?	Short-lived presigned URLs; media bytes bypass PHP.
How does slow work happen?	Valkey queues with separate short, FFmpeg, and Whisper workers.
What is the central data idea?	Review = invitation + access grant + reviewer's work folder.
How are notes timed?	Native TextTrack plus pure <code>computeActive</code> reconciliation; CSS fade.
Why sessions?	Server-side revocation matters for private media.
How is correctness protected?	Policies, validation, transactions, DB constraints, locks, idempotent jobs.
How is it tested?	Vitest/Pest for logic and rules; Playwright for real browser wiring; real-stack media smokes.
What is it not?	Not public, not microservices, not video-in-DB, not one giant queue.

Ask → Check → Store → Queue → Work → Show

Canonical reading in the repository

- `MODERNIZATION_PLAN.md` – detailed architecture and decisions.
- `ARCHITECTURE-EXPLAINED.md` – long-form plain-language explanation.
- `STEPS.md` and `STEP-*.md` – build sequence and acceptance.
- `compose.yaml` and `Dockerfile` – current runtime topology.
- `api/routes/api.php` and `web/src/App.tsx` – current product surface.

Scope note: this guide explains the chosen architecture and the implementation visible through the captions stage. Later roadmap areas—voice annotations, deeper privacy/erasure, admin certification, social connections, production hardening, and accessibility polish—build on these same boundaries. It is a memory aid, not an operational runbook or security audit.